

План разработки Tarot Telegram Bot

Стадия 0. Подготовка проекта

Цель

Зафиксировать архитектуру, стек и структуру репозитория.

Результат

Есть пустой проект с backend, bot, mini app, docker и README.

Делаем

- Создать структуру проекта
- Подключить Docker
- Подготовить `.env`
- Настроить PostgreSQL
- Подготовить базовые конфиги

Промт

Создай структуру проекта для Telegram Tarot Bot по README.

Стек:

- Python
- Aiogram
- FastAPI
- PostgreSQL
- SQLAlchemy / Alembic
- React + TypeScript + Vite
- Docker

Нужна структура папок:

```
backend/  
bot/  
mini-app/  
migrations/  
docs/  
docker-compose.yml  
.env.example  
README.md
```

Пока без бизнес-логики. Только каркас проекта, конфиги и запуск через Docker.

Стадия 1. База данных

Цель

Создать основную схему БД.

Результат

Есть таблицы пользователей, раскладов, карт, трактовок, платежей и истории.

Делаем

- users
- spreads
- readings
- payments
- tarot_cards
- card_interpretations
- card_combinations

Промт

На основе README создай PostgreSQL-схему через SQLAlchemy и Alembic.

Нужны таблицы:

users
spreads
readings
payments
tarot_cards
card_interpretations
card_combinations

Важно:

- readings должен хранить cards_jsonb, tags_jsonb, result_snapshot_jsonb
- у readings должны быть created_at и expires_at
- у spreads должен быть cooldown_hours
- карты и трактовки должны храниться отдельно от раскладов
- предусмотреть deck_code и card_code

Стадия 2. Telegram-бот

Цель

Сделать точку входа пользователя.

Результат

Пользователь запускает бота, регистрируется и видит меню.

Делаем

- `/start`
- регистрация пользователя
- главное меню
- категории раскладов
- переход в Mini App

Промт

Создай Telegram-бота на Aiogram.

Функции:

- команда `/start`
- регистрация пользователя в PostgreSQL
- главное меню
- категории раскладов:
 - Бесплатные
 - Отношения
 - Работа и деньги
 - Будущее и выбор
 - Личное
 - Большие расклады

Кнопка выбранного расклада должна открывать Telegram Mini App.
Пока без оплаты и без генерации расклада.

Стадия 3. Каталог раскладов

Цель

Заложить типы раскладов и ограничения.

Результат

В базе есть все расклады MVP.

Делаем

- название
- код
- категория
- количество карт
- цена

- cooldown_hours
- активность

Промт

Создай seed-данные для таблицы spreads.

Нужны расклады MVP:

Бесплатные:

- card_of_day
- quick_answer
- yes_no

Отношения:

- feelings
- relationship_future
- reconciliation
- hidden_thoughts

Работа и деньги:

- finances
- career
- change_job

Будущее и выбор:

- near_future
- two_options
- month_forecast

Личное:

- cards_advice
- blockers
- hidden_resource

Большие:

- full_relationship
- full_month
- celtic_cross

Для каждого укажи:

code

name

category

description

price

cards_count

cooldown_hours

is_active

Годовой прогноз не добавлять.

Стадия 4. База карт

Цель

Загрузить 78 карт Таро.

Результат

В базе есть все карты с кодами, названиями, score и ссылками на изображения.

Делаем

- tarot_cards seed
- 78 карт
- image_url
- score
- arcana

Промт

Создай seed-файл для 78 карт Таро Райдера – Уэйта.

Для каждой карты нужны поля:

```
deck_code = rider_waite_v1
card_code
name_ru
name_en
arcana
suit
image_url
score
is_active
```

На старте используем только прямые положения карт.
Перевернутые карты не добавлять.

Стадия 5. База трактовок

Цель

Добавить краткие трактовки карт.

Результат

Каждая карта имеет трактовки по темам.

Делаем

- general
- love
- career
- money
- advice
- tags

Промт

Создай базу кратких трактовок для 78 карт Таро.

Для каждой карты нужны трактовки:

- general
- love
- career
- money
- advice

Также нужны:

- tags
- score
- краткое значение карты

Тексты должны быть короткими, понятными, нейтральными и подходить для коммерческого Telegram-бота.

Не копировать тексты из источников дословно.

Формат результата: JSON, готовый для загрузки в PostgreSQL.

Стадия 6. Движок раскладов без ИИ

Цель

Научить систему создавать связный расклад.

Результат

Backend может создать расклад и вернуть JSON результата.

Логика

1. Проверить активный расклад этого типа.
2. Если есть — вернуть старый.

3. Если нет — создать новый.
4. Выбрать карты.
5. Назначить позиции.
6. Посчитать score.
7. Собрать tags.
8. Найти сочетания.
9. Собрать итоговый текст.
10. Сохранить result_snapshot_jsonb.

Промт

Реализуй движок раскладов без ИИ.

Функция `create_or_get_reading(user_id, spread_code)` должна:

1. Проверить, есть ли у пользователя активный reading по этому spread_code, где `expires_at > now()`.
2. Если есть – вернуть существующий reading.
3. Если нет – создать новый.
4. Выбрать случайные карты по `cards_count`.
5. Назначить карты на позиции расклада.
6. Посчитать общий score.
7. Собрать tags.
8. Найти card_combinations.
9. Сформировать result_snapshot_jsonb.
10. Сохранить reading с `expires_at = now() + cooldown_hours`.

Важно:

- один тип расклада нельзя пересоздавать до истечения `expires_at`
- старые расклады не должны меняться
- ИИ не использовать

Стадия 7. Telegram Mini App

Цель

Сделать красивый визуальный расклад.

Результат

Пользователь видит карты, выбирает их, открывает с анимацией и читает результат.

Экраны

1. Старт расклада
2. Перемешивание
3. Выбор карт
4. Переворот карт

- 5. Описание
- 6. Кнопки действий

Промт

Создай Telegram Mini App на React + TypeScript + Vite.

Нужен визуальный интерфейс для Таро-расклада:

Экраны:

1. Старт расклада
2. Анимация перемешивания карт
3. Выбор карт пользователем из 12–15 закрытых карт
4. Переворот выбранных карт
5. Отображение открытых карт
6. Описание расклада под картами

Требования:

- тёмный мистический интерфейс
- без музыки
- плавные CSS / Framer Motion анимации
- карты показывать изображениями
- адаптация под мобильный экран Telegram
- после открытия всех карт показать трактовку

Стадия 8. API для Mini App

Цель

Связать Mini App с backend.

Результат

Mini App получает данные расклада с backend.

Делаем

- endpoint получения расклада
- endpoint выбора карт
- endpoint открытия результата
- проверка Telegram initData

Промт

Создай FastAPI endpoints для Telegram Mini App.

Нужны endpoints:


```
GET /api/spreads
GET /api/readings/{spread_code}
POST /api/readings/{spread_code}/start
POST /api/readings/{reading_id}/select-cards
GET /api/readings/{reading_id}
```

Требования:

- проверять Telegram WebApp initData
- определять пользователя по telegram_id
- если активный расклад уже есть, возвращать его
- если нет, создавать новый через движок
- отдавать карты, позиции, изображения, трактовку и статус reading

Стадия 9. Бесплатный расклад

Цель

Реализовать 1 бесплатный расклад в сутки.

Результат

Пользователь может сделать один бесплатный расклад в день.

Промт

Реализуй бизнес-логику бесплатного расклада.

Правила:

- пользователь может сделать 1 бесплатный расклад в сутки
- тематику выбирает сам
- если бесплатный лимит уже использован, показать платные варианты
- правило неизменности расклада сохраняется
- если пользователь повторно открывает уже сделанный расклад, показывать существующий результат

Добавить поля или таблицу, если нужно для учёта free daily usage.

Стадия 10. Оплата

Цель

Добавить монетизацию.

Результат

Платные расклады доступны после оплаты.

Промт

Добавь оплату для платных раскладов.

Провайдеры:

- Telegram Payments как первый вариант
- архитектурно оставить возможность добавить ЮKassa

Логика:

1. Пользователь выбирает платный расклад.
2. Если активный reading уже есть – открыть его без новой оплаты.
3. Если активного reading нет – показать оплату.
4. После успешной оплаты создать reading.
5. Записать payment в таблицу payments.

Важно:

- не брать повторную оплату за уже активный расклад
- payment должен быть связан с reading_id

Стадия 11. История раскладов

Цель

Дать пользователю доступ к прошлым результатам.

Результат

Есть раздел “История”.

Промт

Реализуй историю раскладов.

Функции:

- список последних раскладов пользователя
- фильтр по категориям
- открытие старого расклада
- отображение старого result_snapshot_jsonb
- кнопка открыть в Mini App

Важно:

- старый расклад должен показываться в том виде, в котором был создан
- новые трактовки не должны автоматически менять старые результаты

Стадия 12. Поделиться раскладом

Цель

Сделать вирусный элемент.

Результат

Пользователь может сгенерировать картинку расклада.

Промт

Реализуй функцию “Поделиться раскладом”.

Нужно:

- создать красивую share-картинку с картами
- показать название расклада
- показать дату
- показать краткий итог
- сохранить `share_image_url` в `readings`
- дать пользователю кнопку отправить / скачать / переслать

Лучший вариант:

- генерировать изображение через Playwright screenshot страницы Mini App
- fallback: генерация через Python Pillow

Стадия 13. Админка

Цель

Управлять проектом без разработчика.

Результат

Админ может менять расклады, цены и тексты.

Промт

Создай простую админку.

Функции:

- список пользователей
- блокировка пользователя
- список раскладов
- изменение цены
- изменение `cooldown_hours`
- включение / отключение расклада

- управление картами
- управление траковками
- управление сочетаниями карт
- просмотр платежей
- базовая статистика

Можно сделать админку как отдельный web-интерфейс на FastAPI или как admin-раздел в Telegram-боте.

Стадия 14. Полировка UX

Цель

Сделать продукт приятным и коммерческим.

Результат

Бот выглядит как готовый сервис, а не тестовый проект.

Делаем

- тексты кнопок
- онбординг
- сообщения об ограничениях
- ошибки оплаты
- пустые состояния
- красивые loading states

Промт

Проведи UX-полировку Telegram-бота и Mini App.

Нужно улучшить:

- тексты меню
- онбординг
- сообщения о повторном раскладе
- сообщения об оплате
- тексты ошибок
- пустую историю
- визуальные состояния загрузки
- кнопки после расклада

Тон:

- мистический
- спокойный
- не пугающий
- без обещаний точного будущего

Стадия 15. Тестирование и релиз

Цель

Подготовить проект к запуску.

Результат

MVP можно отдавать первым пользователям.

Промт

Подготовь проект к MVP-релизу.

Проверить:

- регистрация пользователя
- выбор расклада
- правило активного расклада
- бесплатный лимит 1 раз в сутки
- платные расклады
- оплата
- открытие Mini App
- выбор и переверот карт
- история
- share-картинка
- работа на мобильном Telegram
- ошибки и повторные клики

Добавить:

- README по запуску
- .env.example
- docker-compose production
- базовые тесты
- checklist релиза

Приоритет MVP

Обязательно до запуска

1. База данных
2. Telegram-бот
3. Каталог раскладов
4. База 78 карт
5. Краткие трактовки
6. Движок раскладов
7. Mini App

8. Правило неизменности расклада
9. Бесплатный расклад
10. Оплата
11. История
12. Поделиться

Можно после запуска

- расширенная админка
- больше сочетаний карт
- перевёрнутые карты
- несколько колод
- дополнительные темы оформления
- обновление трактовки старого расклада

Не делаем в MVP

- годовой прогноз
- музыка
- чат с тарологом
- гороскопы
- форум
- уровни пользователей